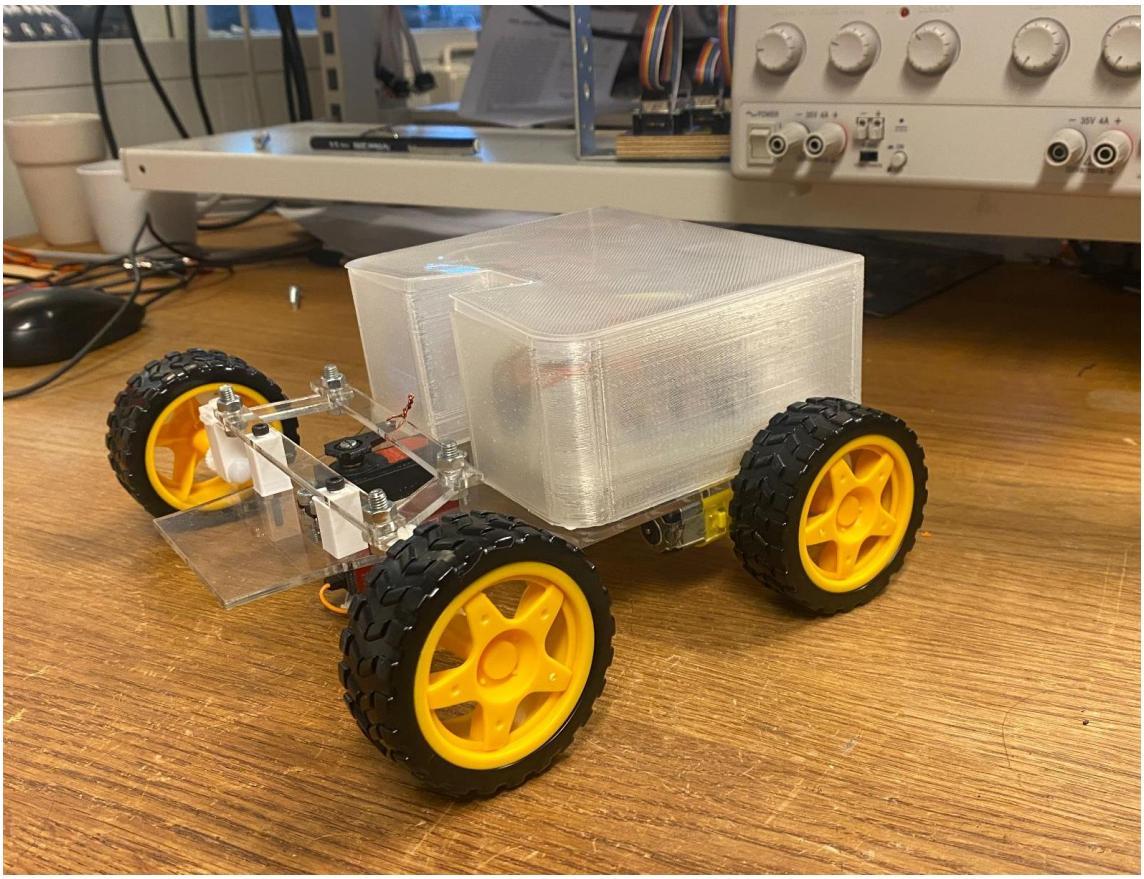
Examensarbete inom teknik, grundnivå, 15 hp
Degree project in Technology, first cycle 15 credits

# Hand Gesture Controlled Car

AMANDA EKHOLM AND SOFIA FRIDELL



Stockholm, Sweden 2024

# Hand Gesture Controlled Car

Amanda Ekholm and Sofia Fridell

Bachelor’s Programme in Design and Product Realization
Date: 2024-06-03
Supervisors: Daniel Frede
Examiner: Daniel Frede
School of Industrial Engineering and Management
Swedish title: Handrörelsestyrd bil

© 2024 Amanda Ekholm and Sofia Fridell

# Abstract

The purpose of this project is to examine how a car controlled by hand gestures wirelessly can be constructed. The hand gestures are measured with a gyro sensor. This project examines how the time delay can be reduced with focus on data transmissions protocols and code structure. Furthermore, the steering is examined so the car can turn while driving. The choice of transmitter was made regarding the latency where nRF24L01 had the lowest latency. To reduce the time delay, the blocking function delay() was avoided which pauses the entire program and instead the millis() function was used, which instead is used to specify how often a code block should run without blocking the rest of the program. Therefore, the Arduino program does not pause, and new gyro signals can be received continually. The time delay was 2.64 ms. The steering is controlled with a servo which is connected to a four-bar linkage inspired by Ackerman's steering principle. To examine the steering the angles of the servo and the front wheels were measured at different angles of the hand.



### Keywords

Mechatronics, wireless data transmission, gyro sensor, servo, Arduino.

## Sammanfattning

Syftet med detta projekt är att undersöka hur en bil som styrs trådlöst med handrörelser kan konstrueras. Handrörelserna mäts med en gyrosensor. I projektet undersöks hur tidsfördröjningen kan reduceras med fokus på val av dataöverföringsprotokoll och kodstruktur. Vidare undersöks hur styrningen ska utformas för att bilen ska kunna svänga i farten. Val av transmitter gjordes med avseende på latens, där nRF24L01 hade lägst latens. För att minska tidsfördröjningen undveks användningen av den blockande funktionen delay() som pausar hela programmet och millis()-funktionen användes istället då den istället används för att specificera hur ofta ett kodblock ska köras utan att blockera resten av programmet. Därmed pausas inte Arduino programmet och nya gyrosignaler kan tas emot kontinuerligt. Tidsfördröjningen blev 2.64 ms. Styrningen görs med en servomotor som är kopplad till en fyrledsmekanism som är inspirerad av Ackermans styrningsprincip. För att utvärdera styrningen, mättes servons och framhjulens vinklar vid olika handvinklar.



### Nyckelord

Mekatronik, trådlös dataöverföring, gyrosensor, servo, Arduino.

## Acknowledgments

During this project we have received valuable support. Firstly, we would like to thank our supervisor Daniel Frede for insightful guidance and help. Thereafter we would also like to thank our lab assistants Kasper Georén, Albin Gunnarsson and Jacob Holst for helping us with our project.

Amanda Ekholm, Sofia Fridell

May 2024

# Table of contents


**1 Introduction ..... 1**
1.1 Background ..... 1
1.2 Purpose ..... 1
1.3 Research Methodology ..... 1
1.4 Delimitations ..... 1
**2 Theory ..... 3**
2.1 Previous work ..... 3
2.2 Steering mechanisms ..... 3
2.3 Microcontroller ..... 4
2.4 IMU ..... 4
2.5 Wireless data transferring ..... 5
2.6 DC motor ..... 6
2.7 H-bridge ..... 7
2.8 Servo motor ..... 7
2.9 Blocking and non-blocking timers ..... 7
**3 Method ..... 9**
3.1 Car Construction ..... 9
3.2 IMU ..... 11
3.3 Data transmission ..... 12
3.4 DC motors ..... 12
3.5 Servo ..... 13
**4 Results and Discussion ..... 15**
4.1 Major results ..... 15
4.2 Discussion ..... 16
**5 Conclusions and Future work ..... 19**
5.1 Conclusions ..... 19
5.2 Future work ..... 19
**6 References ..... 20**
**Appendix A – Code on the hand ..... 22**
**Appendix B – Code on the car ..... 25**
**Appendix C - Draft, Platform ..... 30**
**Appendix D – Draft, Fixed frame ..... 31**
**Appendix E – Draft, Steering arm ..... 32**
**Appendix F – Draft, Drop arm ..... 33**
**Appendix G – Draft, L-attachment ..... 34**
**Appendix H – Draft, Wheel block ..... 35**
**Appendix I – Angle measurements of Steering mechanism ..... 36**

## List of acronyms and abbreviations

| | |
|---|---|
| A/D | Analog to digital |
| CPU | Central Processing Unit |
| DC | Direct current |
| IMU | Inertial Measurement Unit |
| KTH | The Royal Institute of Technology |
| MEMS | Micro-Electromechanical Systems |
| ms | Milliseconds |
| V | Voltage |

# 1 Introduction

## 1.1 Background

The ability to control a miniature car wirelessly is useful since it can be done from a distance, and it is less restrictive since wires can get stuck both in the car itself and in other objects.

When remotely controlling a car, it is important to reduce the time delay from the given instruction to the car's reaction, otherwise the car becomes difficult to handle and thereby increases the risk of collision. The car can be controlled in several different ways. In this project it is done by hand gestures controlling both the speed and direction of the car.

Other hand gesture-controlled projects, such as *Hand Gesture Controlled Omnidirectional Vehicle* [1] and *Hand Gestures Controlled Wheelchair* [2], inspired the decision to create a car that is controlled by hand gestures, but that can turn while driving and where the hand and car is not connected through wires. The hand gestures include tilting the hand upward and downward as well as to the left and right, i.e. along two axes.

## 1.2 Purpose

The purpose of this thesis is to construct a car that is controlled by hand gestures. The following research questions will be examined:

- How can the time delay between the hand gestures and the steering of the car be reduced?
- What is the time delay between the hand gestures and the steering of the car?
- How does the steering mechanism respond to the hand angle?

## 1.3 Research Methodology

The research methodology of the thesis has followed several phases. The first was a literature study where information of the different electronic components and existing types of steering mechanisms were gathered to get a broader knowledge before any practical work. The next phase was to build the prototype of the car by first planning and making decisions about the construction of the car. The next phase was to test the electrical parts on their own to make sure that they were correctly implemented and that they could be used separately. The last phase consisted of integrating the components with each other as well as performing different tests.

## 1.4 Delimitations

For the scope of the project certain delimitations have been made. One of these is that a simplified version of the chosen steering mechanism (i.e. the Ackerman's steering principle) has been implemented. The four-bar linkage and angle of each link have been implemented according to the principle, but these are, however, the only parts of the Ackerman's steering principle that have been incorporated in this thesis. This is because it is not necessary for the scope of this project to have the level of precision that Ackerman's steering principle can provide.

Another delimitation is that front wheel versus rear wheel drive is not examined in the thesis. Rear wheel drive was chosen since this made the construction of the car easier and furthermore

rear wheel drive will lead to lower weight on the front wheels and consequently the requirement of the servo's capacity is lower.

Another delimitation is that only the type of transmitter that is used as well as the code structure will be considered when discussing how to reduce the time delay. However, the quality and capacity of the components can affect the time delay, and this will not be examined as the electronic components that are used in this project are provided by KTH from the mechatronics' lab.

The last delimitation that was made is that the steering mechanism will be evaluated by examining the servo and front wheel angles at different hand angles.

# 2 Theory

## 2.1 Previous work

The idea for this project was inspired by two pervious bachelor's thesis projects at KTH. *Hand Gesture Controlled Omnidirectional Vehicle* [1] and *Hand Gestures Controlled Wheelchair* [2] both controlled the vehicles with hand movements using a combined gyroscope and accelerometer, IMU. *The Hand Gesture Controlled Omnidirectional Vehicle* project controlled their car wirelessly. They had fixed wheel positions and used mechanuum wheels which can create a 360° rotation. The car stands still while the user positions the car in the right direction, the car turns and moves forward separately except when it comes to driving 45° or 90° sideways since the mechanuum wheels make it possible for the car to drive in these directions without turning the car. [1]

*The Hand Gestures Controlled Wheelchair* project controls a wheelchair by transferring data from the IMU through wires. The wheels' positions are also fixed. Each wheel is connected to a stepping motor, when turning the wheelchair, the motors rotate at different speeds which makes it possible for the car to turn while driving forward or backwards. [2]

These projects gave the inspiration to examine how a car could be controlled wirelessly by hand gestures, and to further examine how a car can be steered while driving.

## 2.2 Steering mechanisms

There are several principles to steer a car. The choice of steering mechanism affects for example the accuracy, stability and the possible placement of the car when turning. [3] One method to steer a car is to let the steering axis be fixed and only change the speed of the wheels. If, for example, the right wheel rotates faster than the left, the car will turn to the left. In practice, this can be achieved by having separate motors on the wheels or by using a differential, which is a mechanical component made of e.g. gears that make the different ends of the axis turn at different speeds. [4]

Another principle is to turn the axis of the wheels. To avoid lateral slip of the car (i.e. that the wheels of the car are not only rolling but slipping as well) when applying this principle, every wheel must be perpendicular to their own wheel axis and all wheel axes must intersect at one common point when turning. On four-wheel vehicles usually only the front wheels are steered. This is because of an increase in complexity when steering both the rear and front wheels. [3] One principle for steering the front axis without lateral slip is the Ackerman's steering principle. The front axis is usually a four-bar linkage mechanism [5], but there are also more complex versions [6], which allows the right and left front wheel to rotate different amounts in a turn to help reduce the risk of slippage. The rear wheel axis is fixed and thereby the common center turning point is aligned with the back of the car. When the car is driving straight forward the two drop arms (the linkages between the wheels and control arm) should be at the same angle that intersect at a common point in the middle of the rear wheel axis according to figure 1. [3]

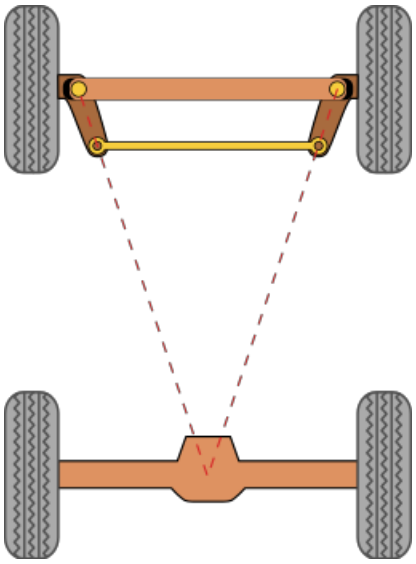


**Figure 1. Ackerman's steering geometry. The two drop arms are placed at an angle so that they align in a common point at the center of the rear wheel axis.** [7]

## 2.3 Microcontroller

A microcontroller is often described as a single chip computer and its main purpose is to control other electrical components. They are used in embedded systems and are commonly found in electrical devices. Microcontrollers are programmable and thereby users can upload their own programs to them. [8] While the program is running, the microcontroller can take input data from a component and create an output to control other components. [9] The components are connected to the microcontroller's ports which have a voltage range between zero to five volts. [10] An A/D converter converts the voltage, which is an analog input, to a digital form. [8] The converting and the sensitivity of the analog input depends on the resolution of the microcontroller, which is given in bits. [11]

After receiving input, the microcontroller's CPU processes the information. [12] [13] A microcontroller also needs a primary memory. There are several different types of primary memories, commonly for primary memories used today is that they can both read and write data. [14] The primary memory has a matrix structure where the different positions have different functions. [15] Information is sent between the different components of the microcontroller through the system bus. [16]

## 2.4 IMU

An IMU is a sensor that is commonly used in electronic devices such as mobile phones and gaming consoles and often these sensors contain both a gyroscope and an accelerometer. The sensor measures the angular velocity of an object which then can be used to determine angular orientation (gyroscope) or change in orientation of the object with respect to a reference axis (accelerometer) [17] according to figure 2. [18] The direction of the reference axis stays unchanged due to the principle of conservation of angular momentum. [19]

There are several types of IMUs that are based on different principles of measuring angular velocity. The most common principle in MEMS is to measure the Coriolis force. The Coriolis force is measured by the help of an oscillating mass. This mass is forced to oscillate in one direction. If

the system rotates, the mass will accelerate in an additional direction than of its initial oscillation and by measuring the force the mass experiences during the new acceleration, the Coriolis acceleration and thereby the angular velocity can be calculated. In MEMS, the digital signal is often retrieved through varying the capacitance depending on the angular velocity of the sensor. [19] IMUs that can detect changes in angular velocity in three axes often use three gyroscopes, one for each axis, and therefore these sensors often require more space and power than single axis gyroscopes. [20]

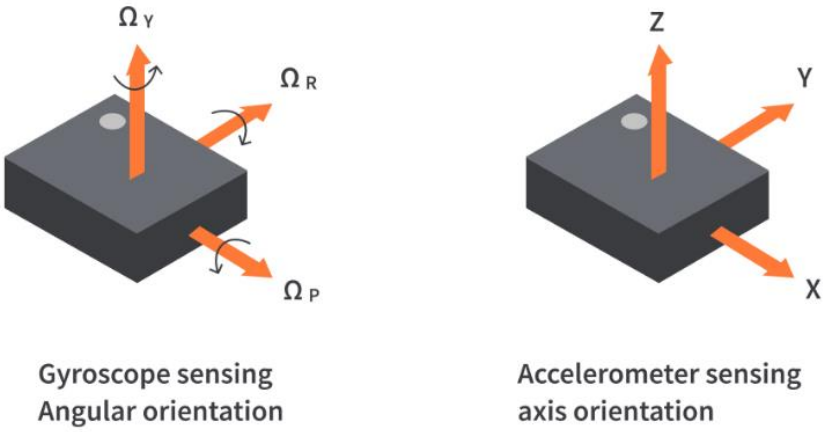



**Figure 2. Difference between gyroscope and accelerometer.** [18]

## 2.5 Wireless data transferring

Most methods for transferring data short distances wirelessly use radio waves to send signals. There are several types of radio wave protocols where Bluetooth, Wi-Fi, ZigBee, Matter, Thread and nRF24 are among the most common today. The properties (such as transmission rate, latency, transmission range, power usage etc.) can vary a lot between different protocols as well as different modules using the same protocol. [21]

In this thesis the focus will be on comparing the latency, i.e. how long it takes for the signal to be sent and received, since one of the research questions is about reducing the time delay between the hand gestures and the steering of the car. In *Benchmarking latency across common wireless links for microcontrollers* [22], the latency for several types of modules were examined (however, only those mentioned above will be discussed in this thesis). Latency varies between different protocols and modules and can also depend a lot on the data payload (the size of the data, usually in bytes). In this project, only small amounts of data will be transferred each loop and therefore the comparison will be made with the results from transferring 12 bytes since that was the smallest payload examined in [22].

Data from the source [22] is presented in table 1 and shows the latency when transferring 12 bytes with different protocols. Both Wi-Fi and Bluetooth are examined by using an ESP32 unit because it's a microcontroller with built in Wi-Fi and Bluetooth that is commonly used in MEMS. ZigBee, Matter and Thread are three different protocols that all use the IEEE 802.15.4 protocol as a basis and will thereby have approximately the same latency and are considered as the one here. To examine the IEEE 802.15.4 protocol an ESP32-C6-MINI was used. To examine the nRF24 protocol an nRF24L01 unit was used. The source [22] describes that the results can vary depending on e.g. settings and optimization efforts. As can be seen in table 1, the protocol with

the lowest latency when transferring 12 bytes is the nRF24. Worth mentioning is that the nRF24 protocol had the lowest latency compared to every protocol and unit mentioned in [22], not only compared to those in the table below.

**Table 1. Latency when transferring 12 bytes with common protocols in MEMS. The table data is from** [22]**.**

| Protocol | Unit | Latency [ms] |
|---|---|---|
| Wi-Fi | ESP32 | 77.2 |
| Bluetooth | ESP32 | 12.9 |
| IEEE 802.15.4 | ESP32-C6-MINI | 2.5 |
| nRF24 | nRF24L01 | 0.3 |

## 2.6 DC motor

The DC motor is an electrical motor that is run with direct current. Figure 3 shows a simplified illustration of a DC motor. It consists of an armature, which is a wire that leads electrical current, that is placed in a magnetic field from a permanent magnet. By applying a voltage over the armature, a current will appear in the wire and give it electromagnetic properties. These properties will cause the armature to rotate until its poles have lined up with the permanent magnet's poles. In a DC motor a commutator with gaps is used to repeatedly change the direction of the current and thereby change the direction of the electromagnetic armature which enables the armature to rotate continuously. This rotation can then be used to mechanically operate and drive a load. In practice, several phase-shifted armatures are used in one motor to obtain a constant angular velocity (if the motor is connected to a DC power source) [23]. Today DC motors come in various sizes and are very common in, for example smaller electronic devices.

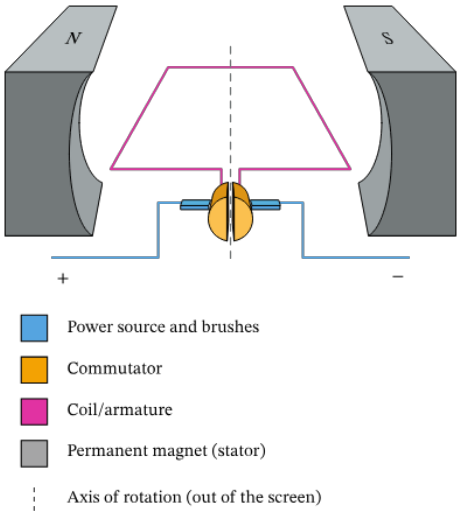



**Figure 3. A simplified illustration of a DC motor with one armature.** [24]

## 2.7 H-bridge

As mentioned in the previous chapter the rotation in a DC motor is created by applying a voltage over the motor. If the polarity over the motor changes, the motor direction will change as well. [25] [26] This can be done by an H-bridge, which is a circuit that has four switches and is connected to a motor as illustrated in Figure 4. The direction of current is dependent on which switches that are open and closed. [27] By pairing the switches diagonally, so that one pair is open and the other pair is closed, the current can run through the motor. By changing which pair is open and which one is closed the current will run in the opposite direction and as a result the voltage polarity over the motor will change as well. [27] [26]

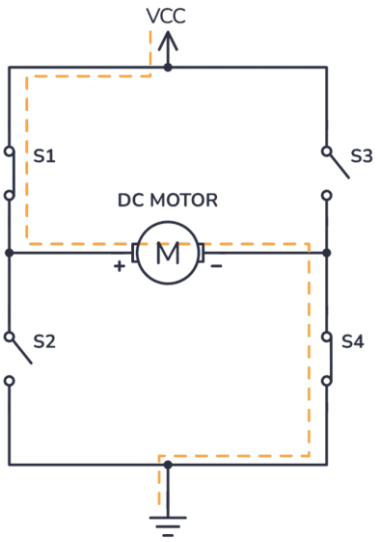



**Figure 4. A simplified illustration of an H-bridge.** [28]

## 2.8 Servo motor

A servo motor can be used for positioning, which is made by a position control loop. [29] When receiving an input signal, the servo motor translates it to a rotation angle. Thereafter the motor will, because of the control loop, take its current position in consideration when adjusting its position. [30]

## 2.9 Blocking and non-blocking timers

Timers are often used in programs for microcontrollers. These can for example be helpful to ensure when or how often a code block is executed. There are several types of timers that work and affect the system performance in different ways. One trait that can affect the performance a lot is if the timer is blocking or non-blocking. A blocking timer blocks the microcontroller's CPU so that it cannot be accessed or used until the code block using the timer is executed. A non-blocking timer on the other hand, does not pause the rest of the program. [31]

An example of a blocking timer is the delay() function which is used to specify how many milliseconds that an Arduino program should pause while running. An example of a non-blocking function is the millis() function which is used to retrieve how many milliseconds that has passed

since the program started. Many times, both functions can be used for the same purpose and achieve similar results, but in some cases the millis() function is considered to be more suitable. This is because of two main reasons. The first is that the millis() function achieves a more accurate timing because it can be used to specify how often a code block should run. The delay() function does not achieve this since the execution time of the code is not included in the delay. The second reason why using millis() can be preferable is that it is non-blocking. It does not pause the entire program as the delay() function does, and therefore other code blocks can still be executed while the code block that depends on the timing from the millis() function is “paused”. [32]

# 3 Method

## 3.1 Car Construction

The car's platform is a rectangle with two one-quarter elliptical cutouts in the front, as can be seen in figure 6. The cutouts were made so the front wheels can turn 45° in each direction without risking colliding with the platform.

Since the car was to be rear wheel driven the DC motors were placed on the back wheels. The motors were attached to the platform's lower surface. This was preferable since it heightens the platform. To be able to attach the motors in the platform, two L formed attachments were designed. The attachments connect to the motor's side and to the lower surface with two screws for each side, see Figure 5. The servo motor was lowered through the platform to reduce the height difference between the four-bar linkage and the wheel axis.

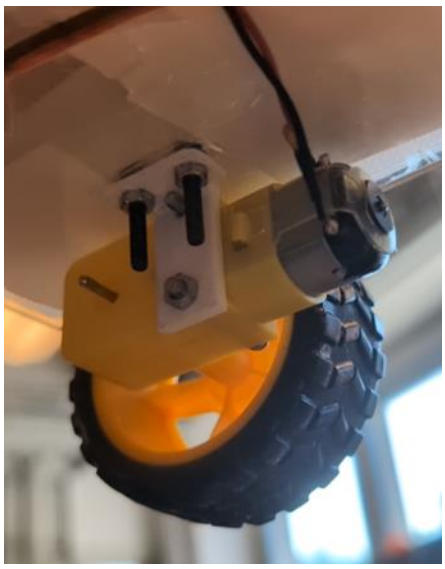


**Figure 5. The L-formed attachment connecting the motor to the platform.**

The steering mechanism that was chosen for the car is a four-bar linkage inspired by Ackerman's steering principle as described in chapter 2.2. The angles of the drop arms are adjusted to meet the requirements of Ackerman's steering principle, see Figure 6. The drop arms are connected to the steering arm which in turn is connected to the servo motor and thereby the drop arms will move when the servo moves.

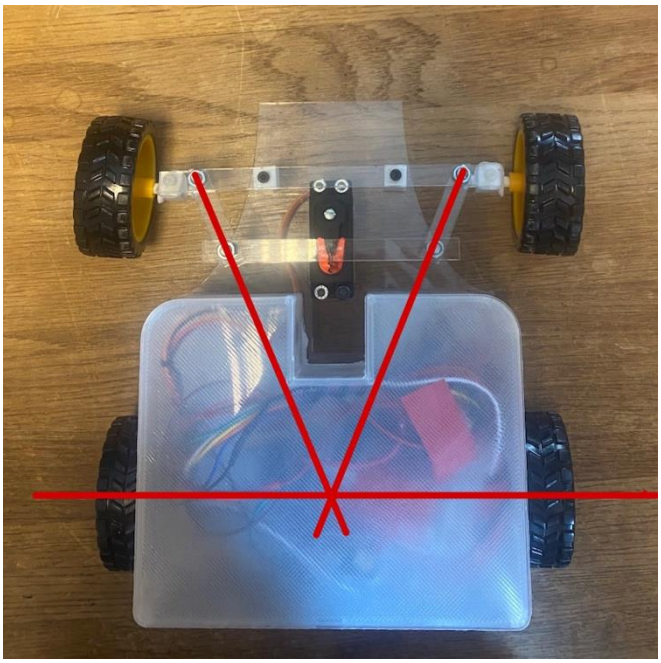


**Figure 6. Construction of the car, showing the common point of the drop arms.**

The servo is lowered through the surface to reduce the level differences between the four-bar linkage and the front wheels.

The drop arms are connected to a fixed frame through an axis which the drop arms can rotate around. The fix frame does however prevent the drop arms from moving sideways. The drop arms are each connected to a block that has a hole in it, according to Figure 7. The block also has a horizontal hole that aligns with the center point of the wheels. To connect the block and the wheel, a wheel axis goes from the shaft of the wheel and through the hole. The part of the wheel axis that goes through the block hole is round and slightly smaller than the hole and thereby the wheel axis can spin freely. The wheel axis design is from [33].



**Figure 7. The wheel block that connects the drop arm to the wheel through a wheel axis.**

See Appendix C-H for the drafts of each part of the car construction.

To attach and stabilize the components on the hand, a platform was made. The platform is then attached to a ribbon which can be placed around the hand. The components of the hand needed to be powered by 6 V, but to make the construction of the hand both smaller and lighter, a 9 V battery was used instead of using four 1.5 V batteries. To adjust the 9 V battery to 6 V, a step-down module LM2596, which is a switch regulator, was connected to the battery according to [34].

To be able to turn on and off the car, a toggle switch was attached to the battery of the hand according to [35]. This made it possible to turn on and off the car without disconnecting the battery since the car is programmed to stop if it does not receive any new signals from the hand.

## 3.2 IMU

The IMU that is used is the MPU6050 which has a three-axis gyroscope and a three-axis accelerometer. It is placed on a breadboard and connected to the Arduino on the hand according to [36]. The code is based on the code from [36], and calculates the angles $\theta_x$, $\theta_y$ and $\theta_z$ from the gravitational acceleration in each axis $a_x$, $a_y$ and $a_z$ with trigonometry:

$$\theta_x = \tan^{-1}\left(\frac{a_y}{a_z}\right)$$

$$\theta_y = \tan^{-1}\left(\frac{a_x}{a_z}\right)$$

$$\theta_z = \tan^{-1}\left(\frac{a_y}{a_x}\right)$$

Only the two angles $\theta_x$ and $\theta_y$ are used in this project whereas the $\theta_x$ is used to determine the speed of the car and $\theta_y$ is used to determine the direction. When the hand is flat both angles are at

0°. They have an angle span from 0-360° and to make further calculations easier, the angle spans are recalculated so that they instead go between -90° and +90°. Angles in a span of +5° and -5° to 0° are regarded as 0°. Then the angle ratio or percentage is calculated according to:

$$angle\ ratio = \frac{\theta}{90}$$

### 3.3 Data transmission

Regarding the analysis from the theory, the nRF24 protocol was chosen as the best option to be used for transmitting data wirelessly in this project. The unit nRF24L01+ is used which can act both as a transmitter and a receiver and thereby two of the same units are used - one on the hand and another on the car. In this project, two data points are transmitted simultaneously in each loop of the Arduino. The first is a variable that represents what speed the car should have and the other represents the direction i.e. if the car is to turn in either direction.

The code that is used is in Appendix A and B and has its basis from [37], but has been modified. The transmitter and receiver are connected to the Arduino on the hand and the car respectively, according to [38].

### 3.4 DC motors

The two DC motors are connected to the Arduino Uno board on the car, an H-bridge L298 motor driver module and a 9-voltage battery according to [39]. When the receiver first receives a speed signal the motors turn on, thereafter every new speed signal is compared with the previous signal to determine if it is the same speed as before or if the motors are to accelerate or brake and, if that is the case, it changes the speed based on the current hand angle.

The speed interval is between -90° to 90°, where 90° is the top speed forward and the -90° is the top speed backwards. The speed is based on the ratio between the current hand angle to maximum hand angle, the ratio is multiplied with 255, which is the maximum number of bits, to receive the corresponding speed.

If the hand's angle changes from a positive to a negative angle or the other way around, the program calls a function that changes the direction of the car by changing which switches that are opened on the H-bridge.

The Arduino code written for the motors is based on a code from [39]. It has thereafter been modified to adjust the speed and direction of the motors based on the signal of the IMU.

The DC motors do not react to small angles from the IMU because the voltage gets too small to drive the motors. To make them start spinning for small hand angles, the speed ratio is rescaled with an offset, which can be compared to the normalization process according to [40]. In the code this is done with an offset of 35% of 255 according to the formula:

$$rescaled\ speed\ ratio = \frac{speed\ ratio \cdot 255 + 0.35 \cdot 255}{255 + 0.35 \cdot 255}$$

### 3.5 Servo

For the steering of the car, a QY3225mg servo is used. The servo is connected to an Arduino Uno board and a 6 V battery according to [41]. A ratio between the current hand angle and the maximum angle is calculated. One direction is set to a positive direction and the other direction is set to a negative direction. Therefore, the ratio will be either positive or negative depending on the turning direction of the car.

The ratio is sent to the car. The servo thereafter changes its position by multiplying the servo's starting position with the ratio and thereafter adding it to the starting position. The servo is set to go between -50° to +50°.

The Arduino code is based on example code found in the Arduino IDE program, Examples, Arduino uno example, servo, sweep. Thereafter it has been modified to adjust its position based on the angle of the IMU and the angle range was changed to fit the car's steering range.

# 4 Results and Discussion

## 4.1 Major results

To measure the time delay between the hand gestures and the movement of the car, both Arduinos were connected to an oscilloscope. Two pulses were created by setting a pin on each Arduino to HIGH for one millisecond. The first pulse was placed before the IMU signal was registered on the hand and the second pulse was placed after the car had received the signal and could use it. The oscilloscope could then detect the pulses and the time difference between them were measured. The time delay was measured one time.

**Table 2. The time delay from a hand gesture to the controlling of the car**

| Time delay [ms] |
|---|
| 2.64 |

To examine how the steering responded to the hand angle, the angles of the servo's position and the two front wheels were measured. The Arduino on the hand was connected to a computer to be able to read the hand angles. By measuring the servo's position, it was possible to review how the servo corresponded with the hand angle. The test was repeated for 25, 50, 75 and 100 percent of the maximum hand angle in one direction. To examine the implementation of the steering mechanism it was compared to a theoretically calculated value of the angle of the inner wheel, which here is the left wheel $\theta_2$ according to Ackerman's steering principle. This was calculated from the measured value of the outer wheel, which here is the right wheel angle $\theta_1$ according to the formula [5]:

$$\theta_2 = \operatorname{arccot}\left(cot\theta_1 - \frac{b}{l}\right)$$

where $b$ is the distance between the front wheels and $l$ is the distance between the front and back wheel axes. The test was repeated three times for each angle. In Table 3 the mean value of the angles is presented. In Appendix I the result from each measurement is presented. This to examine how the servo and wheels responded to different hand angles.

**Table 3. Resulting angles for different percentage of maximal hand angle (mean value).**

| Percentage of maximal hand angle [%] | Servo angle in code [°] | Measured servo angle [°] | Angle right front wheel [°] | Angle left front wheel [°] | Theoretical angle of left wheel given the right wheel angle [°] |
|---|---|---|---|---|---|
| 25 | 12.5 | 13.13 | 2.67 | 1.83 | 2.86 |
| 50 | 25 | 25.68 | 8.17 | 8.33 | 10.22 |
| 75 | 37.5 | 38.62 | 12.5 | 16 | 17.93 |
| 100 | 50 | 49.07 | 15.33 | 22 | 24.17 |

## 4.2 Discussion

The time delay between the hand gestures and the controlling of the car is 2.64 ms, as can be seen in table 2. This is not noticeable when controlling the car. This is somewhat higher than the latency that was presented in table 1. However, the measured time delay also includes registering and retrieving signals from the IMU as well as the calculations made before the information is sent with the nRF24L01+ unit. Therefore, it is plausible that the measured time delay is higher than the latency, since the latency presented in Table 1 only measures the time between sending the signal to receiving it. To get a more reliable result for the time delay, the measurement could have been conducted more than one time.

As mentioned in section 3.2 Data Transmission, the nRF24 protocol was chosen to minimize the time delay when transmitting the data. The current time delay is smaller than the latency for both the Bluetooth and WiFi protocol, and therefore the choice of transmitter and receiver has had a significant impact on the result.

Another factor that has had a noticeable effect on the time delay is the structure of the code on the Arduino of the car. The first version of the code received a signal from the nRF24L01+ unit and then used a for-loop with delays to step by step change the speed of the motors to the desired speed. While the motors changed speed, the nRF24L01+ unit on the hand transmitted new signals that the nRF24L01+ unit on the car never used because the main loop was not finished. Only when the main loop finally started over, a new signal could be retrieved and used. The new version of the code uses the non-blocking millis() function instead of blocking delays. The code is set to update the speed of the motors every 20 ms and uses every signal that is transmitted and received instead of waiting for the motors to change first and then using the first signal that is available when done.

The decision to use millis() has also helped with making the servo and motor move more simultaneously since the delay has been reduced significantly. This decision reduced the time delay noticeably by several seconds, however no exact time data was saved.

When comparing the measured angle of the servo with the angle set in code, a slight difference can be seen. The results show that the servos maximum angle is not 50° but 49.07°. The highest difference between the hand angle and the servo is when the hand angle is at 37.5°, then the servos angle differs with 1.12°, which may be caused by error of measurements. Furthermore, the

difference in the maximum turning range is an explanation to why the other measured angles also differ from the expected angles.

The two front wheels have different angles for each position, this is expectable since the steering is made with a four-bar linkage. The inner wheel (in this case the left front wheel) should always have a higher angle than the outer wheel. However, as can be seen in table 3, the result shows that this was not the case for the smallest angle.

A possible explanation for this is error of measurement when measuring the wheel angles because it was made by hand and for small angles the error of measurement will have a bigger impact. Another explanation is that the wheel axes of the front wheels are not perfectly fitted because they need to be able to rotate freely. These parts were 3D printed which lowers the precision. To improve this bearings could be used instead. Another source of error is that the IMU signal is sensitive to changes. This is noticeable when holding the hand at an angle since it is difficult to hold the hand completely steady. However, small changes in the IMU signal have an insignificant effect on the car and therefore this is not noticeable when driving the car.

The theoretical angles of the left front wheel are similar to the actual measured angles, especially for smaller angles. The biggest difference is 2.17°, which is for the largest angle. The results show that a simplified version of Ackerman's steering principle still gives similar results when it comes to the wheel angles that a real Ackerman's steering does. However, the four-bar linkage and the wheel angles are the only things that have been implemented and compared to a real Ackerman's steering. A possible explanation to why the results differ from the theoretical values is that the distance between the front wheels were not considered when designing the car. This value as well as the distance between the front and back wheel axes, are used in the formula for the theoretical wheel angles and will thereby affect the results.

The evaluation of the steering has been done in two steps, first how the servo responds to the hand angle and then evaluate how the steering mechanism compares to the theoretical values of Ackerman's steering based on wheel angles. However, the wheel angles are not mapped directly to the hand angle. A possible improvement could have been to further examine how the two could be connected.

Even though there are possibilities for improvement, these possible adjustments are not necessary for the scope of this project since the car moves as expected when driving the car.

# 5 Conclusions and Future work

## 5.1 Conclusions

The purpose of this project was to build a car that was controlled by hand gestures and examine the steering and how to reduce the time delay.

The time delay between the hand gestures and the steering of the car is 2.64 ms. By choosing a transmitter with low latency when sending small amounts of data and by the structure of the code, the time delay could be reduced significantly.

The steering mechanism responds sufficiently to the hand angle for the scope of this project. The hand angles, servo angle and wheel angles follow a similar pattern, but do not always correspond exactly to each other as expected.

## 5.2 Future work

If the project would have continued, a more extensive time analysis could be made to further reduce the time delay. This could be done by measuring the time delay for each component to determine where the focus for further efforts to reduce the time delay ought to be made.

A higher precision of the steering mechanism could be made by fully implementing Ackerman's steering principle. As mentioned in the discussion, a possible improvement would be to evaluate the expected wheel angles in regard to the hand angle as well as the other wheel angle to get a better understanding of the implementation of the steering. Lastly, by increasing the angle range of the servo the maximum angle of the wheels could be higher.

# 6 References


[1] K. N. A. Beckman, "Hand Gesture Controlled Omnidirectional Vehicle," Stockholm, 2020.

[2] R. N. Almida Winquist De Val, "Hand Gestures Controlled Wheelchair," Stockholm, 2019.

[3] K. N. W. S. T.K. Garrett, "Chapter 40 - Front axle and steering mechanism," in *Motor Vehicle (13)*, 2000, pp. 1043-1045.

[4] Universal Technical Institute, "What is a Differential?," 11 08 2021. [Online]. Available: https://www.uti.edu/blog/automotive/differential. [Accessed 04 03 2024].

[5] X. L. Z.-J. F. a. J. S. D. Jing-Shan Zhao, "Design of an Ackermann-type steering mechanism," *Journal of Mechanical Engineering Science,* vol. 227, no. 11, 2013.

[6] M. A. C. N. B. D, "Study regarding the implementation of an Ackerman steering," *IOP Conference Series: Materials Science and Engineering,* vol. 568, 2019.

[7] "Designing an Ackermann steering mechanism," [Online]. Available: https://mechanics.stackexchange.com/questions/13499/designing-an-ackermann-steering-mechanism. [Accessed 16 04 2024].

[8] D. Ibrahim, "CHAPTER 1 - Microcontroller System," in *SD Card Projects Using the PIC Microcontroller*, Newes, 2010, pp. 8-9.

[9] New York University, "NYU," [Online]. Available: https://itp.nyu.edu/physcomp/lessons/microcontrollers-the-basics/. [Accessed 24 02 2024].

[10] M. Paulson, "Mikrodatorn," in *Elektroteknik*, KTH, Institutionen för Maskinkonstruktion, Mekatronik, 2013, pp. 9-17-9-18.

[11] M. Paulson, "Mikrodatorn," in *Elektroteknik*, KTH, Institutionen för Maskinkonstruktion, Mekatronik, 2013, pp. 9-17.

[12] M. Paulsson, "Mikrodatorn," in *Elektroteknik*, KTH, Institutionen för Maskinkonstruktion, Mekatronik, 2013, pp. 9-5.

[13] D. Ibrahim, "CHAPTER 1 - Microcontroller System," in *SD Card Projects Using the PIC Microcontroller*, Newnes, 2010, p. 4.

[14] M. Paulson, "Mikrodatorn," in *Elektroteknik*, KTH, Instutitionen för Maskinkonstruktion, Mekatronik, 2013, pp. 9-10 - 9-13.

[15] M. Paulson, "Mikrodatorn," in *Elektroteknik*, KTH, Institutionen för Maskinkonstruktion, Mekatronik, 2013, pp. 9-10.

[16] M. Paulson, "Mikrodatorn," in *Elektroteknik*, KTH, Institutionen för Maskinkonstruktion, Mekatronik, 2013, pp. 9-3.

[17] Analog Devices, "www.analog.com," 17 03 2015. [Online]. Available: https://www.analog.com/en/resources/technical-articles/accelerometer-and-gyroscopes-sensors-operation-sensing-and-applications.html. [Accessed 08 05 2024].

[18] CircuitBread, "How do Accelerometers and Gyroscopes work?," 02 02 2022. [Online]. Available: https://www.circuitbread.com/ee-faq/how-do-accelerometers-and-gyroscopes-work. [Accessed 08 05 2024].

[19] G. V. C. Z. L. Allegato, "Chapter 46 - Gyroscopes," in *Handbook of Silicon Based MEMS Materials and Technologies (Third Edition)*, 2020.

[20] V. K. R. Pradnya Chabbi, "Design and fabrication of single drive tri-axis MEMS gyroscope," *Measurement,* vol. 227, 2024.

[21] J. C. L. Lawrence J. Henschen, "Chapter 23 - Low-level communication protocols," in *Embedded System Design - Methodologies and Issues*, 2024, pp. 437-467.

[22] S. Rapson, "Benchmarking latency across common wireless links for microcontrollers," Electricui, [Online]. Available: https://electricui.com/blog/latency-comparison#results. [Accessed 16 04 2024].

[23] H. M. R. Johansson, "Elmotordrift - 7.4 Likströmsmotorn," in *Elektroteknik*, KTH, Institutionen för Maskinkonstruktion, Mekatronik, 2013, pp. 7-8 - 7-10.

[24] "nagwa.com," NAgwa, [Online]. Available: https://www.nagwa.com/en/explainers/246108560531/. [Accessed 16 04 2024].

[25] "MF1016 Föreläsning 7, Mikrokontroller i inbyggda system," KTH, [Online]. Available: https://canvas.kth.se/courses/41991/pages/mikrokontrollern-i-inbyggda-system-f7-09?module_item_id=712347. [Accessed 06 Mars 2024].

[26] New York University, "NYU," [Online]. Available: https://itp.nyu.edu/physcomp/lessons/dc-motors-the-basics/#Direction. [Accessed 30 May 2024].

[27] "MF1016 Föreläsning 5 - Varvtalstyrning av likströmsmotorer," KTH, [Online]. Available: https://canvas.kth.se/courses/41991/pages/varvtalstyrning-av-likstromsmotorer-f5-07?module_item_id=712337. [Accessed 06 Mars 2024].

[28] Ø. N. Dahl, "Build electronic Circuits," 5 December 2018. [Online]. Available: https://www.build-electronic-circuits.com/h-bridge/. [Accessed 2024 April 25].

[29] H. S. M. H. Y. K. Pakorn Serikitkankul, "Current Limiter Complicates The Synamic Characteristics of Servo Motor," in *Mechatronics for Saftey, Security and Dependability in a New Era*, Department of Mechanical Systems Engineering, Kanazawa University, 2007, pp. 329-332.

[30] H. Liu, "Chapter 3 - Rail tranist collaborative robot systems," in *Robot Syatems for Rail Transit Applications*, Elsevier, 2020, p. 119.

[31] mikesims10670, "Non-Blocking vs. Blocking Arduino Code," 26 02 2023. [Online]. Available: https://mikesims10670.livejournal.com/11316.html?noscroll&utm_medium=endless_scroll. [Accessed 29 05 2024].

[32] M. Aasvik, "Arduino Tutorial: Using millis() Instead of delay()," [Online]. Available: https://www.norwegiancreations.com/2017/09/arduino-tutorial-using-millis-instead-of-delay/. [Accessed 01 05 2024].

[33] "Printables," 4 November 2022. [Online]. Available: https://www.printables.com/model/309665-dc-motor-with-148-gear-3-6v-with-double-sided-shaf/files. [Accessed 9 april 2024].

[34] P. GEN, "Youtube - LM 2596 DC -DC Buck converter | Adjustable step down module | specifications -Applicatio," 20 Juni 2018. [Online]. Available: https://www.youtube.com/watch?v=QPntXt8Ea3s. [Accessed 26 april 2024].

[35] "DIYSIM," [Online]. Available: https://www.737diysim.com/wiring-a-switch-to-arduino. [Accessed 30 April 2024].

[36] A. Burkay KIRNIK, "How to Measure Angle With MPU-6050(GY-521)," [Online]. Available: https://www.instructables.com/How-to-Measure-Angle-With-MPU-6050GY-521/. [Accessed 21 03 2024].

[37] t. hibit, "How to use the NRF24L01 module with Arduino," 09 10 2022. [Online]. Available: https://projecthub.arduino.cc/tmekinyan/how-to-use-the-nrf24l01-module-with-arduino-813957 . [Accessed 22 03 2024].

[38] Robin2, "Simple nRF24L01+ 2.4GHz transceiver demo," 06 2016. [Online]. Available: https://forum.arduino.cc/t/simple-nrf24l01-2-4ghz-transceiver-demo/405123. [Accessed 21 03 2024].

[39] "Last Minute Engineers," [Online]. Available: https://lastminuteengineers.com/l298n-dc-stepper-driver-arduino-tutorial/. [Accessed 22 mars 2024].

[40] Codecademy Team, "Normalization," [Online]. Available: https://www.codecademy.com/article/normalization. [Accessed 29 05 2024].

[41] "Circuit Basics," [Online]. Available: https://www.circuitbasics.com/controlling-servo-motors-with-arduino/. [Accessed 27 mars 2024].

# Appendix A – Code on the hand

```
//**********Sources*****************
//The basis of the IMU code is taken from
//https://www.instructables.com/How-to-Measure-Angle-With-MPU-6050GY-521/
//and written by Ahmet Burkay KIRNIK, but has been modified by Amanda Ekholm and Sofia Fridell

//The basis of the transmitter (nRF24L01) code is taken from
//https://projecthub.arduino.cc/tmekinyan/how-to-use-the-nrf24l01-module-with-arduino-813957
//and is written by tmekinyan and hibit, but has been modified by Amanda Ekholm and Sofia Fridell
//**********************************

//Measure Angle with a MPU-6050(GY-521)
// IMU
#include<Wire.h>
const int MPU_addr=0x68;
int16_t AcX,AcY,AcZ,Tmp,GyX,GyY,GyZ;
int minVal=265;
int maxVal=402;
float x;
float y;
float z;
float speed_ratio;
float steering_ratio;

//transmitter
#include "SPI.h"
#include "RF24.h"
#include "nRF24L01.h"
#define CE_PIN 9
#define CSN_PIN 10
#define INTERVAL_MS_TRANSMISSION 100
RF24 radio(CE_PIN, CSN_PIN);
const byte address[6] = "13713";
//NRF24L01 buffer limit is 32 bytes (max struct size)
struct payload {
  short speed;
  short steering;
};
payload payload;

void setup() {
 // IMU
 Wire.begin();
 Wire.beginTransmission(MPU_addr);
 Wire.write(0x6B);
 Wire.write(0);
 Wire.endTransmission(true);
 Serial.begin(115200);
```

```
  pinMode(2, OUTPUT);

  // transmitter
  radio.begin();
  //Append ACK packet from the receiving radio back to the transmitting radio
  radio.setAutoAck(true); //(true|false)
  //Set the transmission datarate
  radio.setDataRate(RF24_2MBPS); //(RF24_250KBPS|RF24_1MBPS|RF24_2MBPS)
  //Greater level = more consumption = longer distance
  radio.setPALevel(RF24_PA_MAX); //(RF24_PA_MIN|RF24_PA_LOW|RF24_PA_HIGH|RF24_PA_MAX)
  //Default value is the maximum 32 bytes
  radio.setPayloadSize(sizeof(payload));
  //Act as transmitter
  radio.openWritingPipe(address);
  radio.stopListening();
}

void loop() {
  // IMU
  Wire.beginTransmission(MPU_addr);
  Wire.write(0x3B);
  Wire.endTransmission(false);
  Wire.requestFrom(MPU_addr,14,true);
  AcX=Wire.read()<<8|Wire.read();
  AcY=Wire.read()<<8|Wire.read();
  AcZ=Wire.read()<<8|Wire.read();
  int xAng = map(AcX,minVal,maxVal,-90,90);
  int yAng = map(AcY,minVal,maxVal,-90,90);
  int zAng = map(AcZ,minVal,maxVal,-90,90);

  //calculates the angles from the accelerations
  x= RAD_TO_DEG * (atan2(-yAng, -zAng)+PI);
  y= RAD_TO_DEG * (atan2(-xAng, -zAng)+PI);
  z= RAD_TO_DEG * (atan2(-yAng, -xAng)+PI);


    //code to control speed (the motors)
    //recalculates the upper angle span from 180 to 360 to a negative scale from -90 to 0
    if (x > 180) {
      x = x - 360;
      //the minimum angle is set to -90
      if (x < -90) {
        x = -90;
      }
    //the maximum angle is set to +90
    } else if (x <= 180 && x > 90) {
      x = 90;
    }
    //if within the interval -5 to +5 degrees, set x to 0 degrees
```

```
  if (x >= -5 && x <= 5) {
    x = 0;
  }
  speed_ratio = x/90;

  //code to control steering (the servo)
  //recalculates the upper angle span from 180 to 360 to a negative scale from -90 to 0
  if (y > 180) {
    y = y - 360;
    //the minimum angle is set to -90
    if (y < -90) {
      y = -90;
    }
    //the maximum angle is set to +90
  } else if (y <= 180 && y > 90) {
    y = 90;
  }
  //if within the interval -5 to +5 degrees, set x to 0 degrees
  if (y >= -5 && y <= 5) {
    y = 0;
  }
  steering_ratio = y/90;

  // transmitter
  //convert to short with four digits of significance (to send a short instead of float)
  payload.speed = speed_ratio*1000;
  payload.steering = steering_ratio*1000;

  radio.write(&payload, sizeof(payload));

  Serial.print("speed_ratio:");
  Serial.println(payload.speed);
  Serial.print("steering_ratio:");
  Serial.println(payload.steering);

  Serial.println("Sent");


  delay(INTERVAL_MS_TRANSMISSION);
}
```

## Appendix B – Code on the car

```
//**********Sources*****************
//The basis of the receiver (nRF24L01) code is taken from
//https://projecthub.arduino.cc/tmekinyan/how-to-use-the-nrf24l01-module-with-arduino-813957
//and is written by tmekinyan and hibit, but has been modified by Amanda Ekholm and Sofia Fridell
//The basis of the DC motor and H-bridge code is taken from https://lastminuteengineers.com/l298n-dc-stepper-driver-arduino-tutorial/
//but has been modified by Amanda Ekholm and Sofia Fridell
//The basis of the servo code is taken from Arduino IDE program, Examples, Arduino uno example, servo, sweep.

// but has been modified by Amanda Ekholm and Sofia Fridell
//**********************************

#include "SPI.h"
#include "RF24.h"
#include "nRF24L01.h"
#define CE_PIN  2
#define CSN_PIN 10
#define INTERVAL_MS_SIGNAL_LOST 5000
#define INTERVAL_MS_SIGNAL_RETRY 100
#define INTERVAL_MOTOR 20
RF24 radio(CE_PIN, CSN_PIN);
const byte address[6] = "13713";
//NRF24L01 buffer limit is 32 bytes (max struct size)
struct payload {
   //byte speed;
   //char steering;
   short speed;
   short steering;

};
payload payload;
unsigned long lastSignalMillis = 0;
unsigned long lastSpeedChange = 0;


//servo
#include <Servo.h>

Servo myservo;  // create servo object to control a servo
int pos = 0;    // variable to store the servo position
int angle_range = 80;
int stop_interval = 5;
float new_angle;
float prev_angle;
```

```
//motor
// Motor A connections
int enA = 6;
int in1 = 8;
int in2 = 7;
// Motor B connections
int enB = 3;
int in3 = 5;
int in4 = 4;

//speed
int state;
float speed;
float prev_speed = 0;
float steering;
float normalization_ratio;

bool radioRecievedStatus;
bool radioBeginStatus;


void setup()
{
   Serial.begin(115200);
   radioBeginStatus = radio.begin();
   if (!radioBeginStatus) {
    Serial.println("Radio hardware not responding");
   }
   //Append ACK packet from the receiving radio back to the transmitting
radio
   radio.setAutoAck(true); //(true|false)
   //Set the transmission datarate
   radio.setDataRate(RF24_2MBPS); //(RF24_250KBPS|RF24_1MBPS|RF24_2MBPS)
   //Greater level = more consumption = longer distance
   radio.setPALevel(RF24_PA_MIN);
//(RF24_PA_MIN|RF24_PA_LOW|RF24_PA_HIGH|RF24_PA_MAX)
   //Default value is the maximum 32 bytes1
   radio.setPayloadSize(sizeof(payload));
   //Act as receiver
   radio.openReadingPipe(0, address);
   radio.startListening();



   //motor
   // Set all the motor control pins to outputs
```

```
  pinMode(enA, OUTPUT);
  pinMode(enB, OUTPUT);
  pinMode(in1, OUTPUT);
  pinMode(in2, OUTPUT);
  pinMode(in3, OUTPUT);
  pinMode(in4, OUTPUT);

  // Turn off motors - Initial state
  digitalWrite(in1, LOW);
  digitalWrite(in2, LOW);
  digitalWrite(in3, LOW);
  digitalWrite(in4, LOW);

  //servo
    myservo.attach(9);  // attaches the servo on pin 9 to the servo object

    //myservo.write(angle_range/2); //position the servo in the middle
    for (pos = 0; pos <= 80/2; pos += 1) { // goes from 0 degrees to 180
degrees
      // in steps of 1 degree
      myservo.write(pos);              // tell servo to go to position in
variable 'pos'
      delay(5);
    }
    prev_angle = pos;
}
void loop()
{
   radioRecievedStatus = radio.available();
   unsigned long currentMillis = millis();
   if (radio.available() > 0) {
     radio.read(&payload, sizeof(payload));

     lastSignalMillis = currentMillis;
     speed = payload.speed/1000.0;
     steering = payload.steering/1000.0;

     //servo
     new_angle = steering * angle_range/2;
     pos = angle_range/2 + new_angle;
     myservo.write(pos);
   }

   if (currentMillis - lastSignalMillis > INTERVAL_MS_SIGNAL_LOST) {
     myservo.write(angle_range/2);
     //motor
     speedControl(0);
```

```
      prev_speed = 0;
      //lostConnection();
    }

    if (currentMillis - lastSpeedChange > INTERVAL_MOTOR) {
     //motorstyrning
     if (speed <= 0 && prev_speed > 0) {
       setForward();
     } else if (speed > 0 && prev_speed <= 0) {
       setBackward();
     }

     speedControl(abs(speed));

     lastSpeedChange = currentMillis;
     prev_speed = speed;
    }
}


void lostConnection()
{
   Serial.println("We have lost connection, preventing unwanted behavior");
   delay(INTERVAL_MS_SIGNAL_RETRY);
}

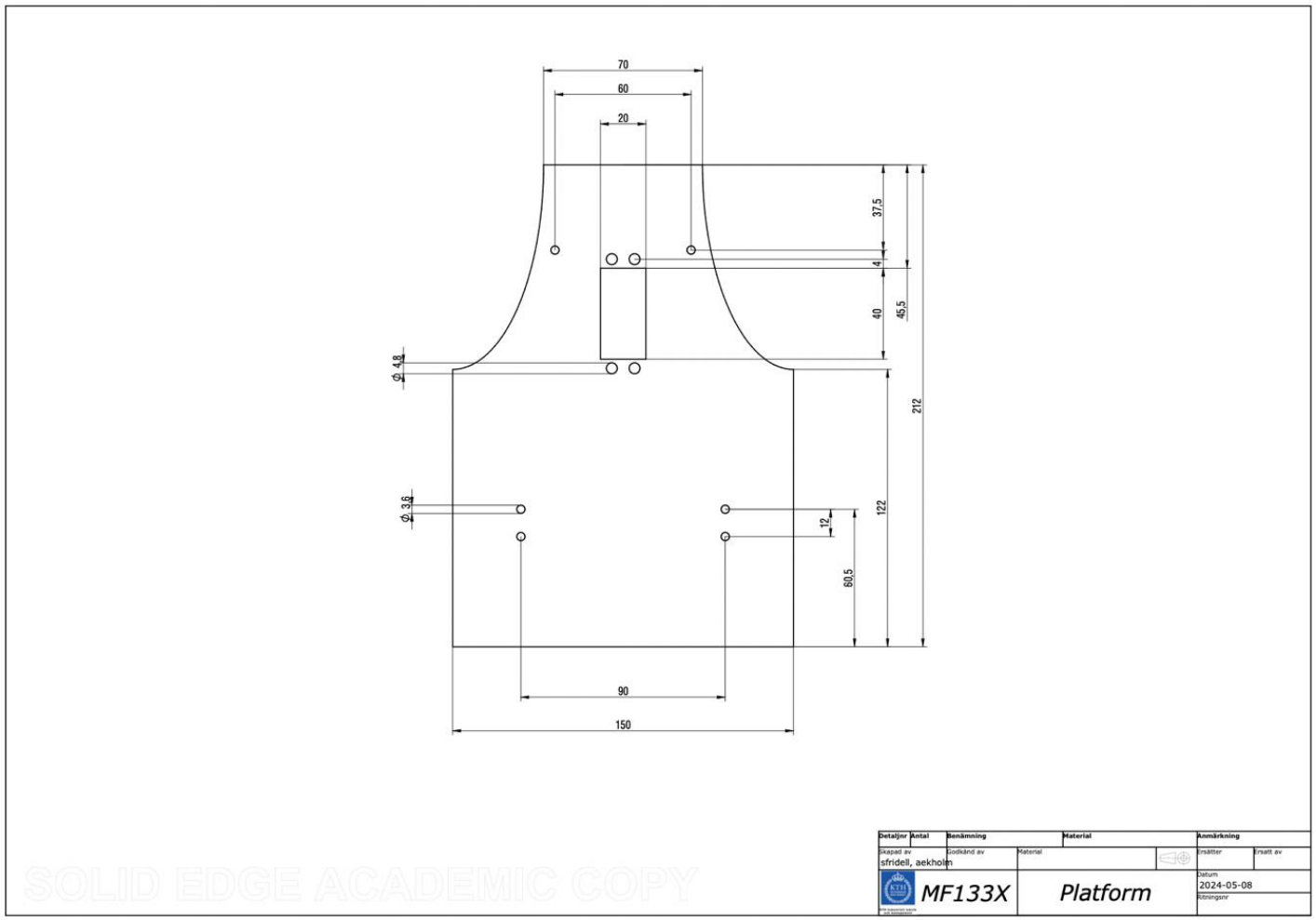
//motor
void speedControl(float new_speed) {
  normalization_ratio = (new_speed*255 + 255*0.35)/(255 + 255*0.35);

  analogWrite(enA, normalization_ratio*255);
  analogWrite(enB, normalization_ratio*255);
}

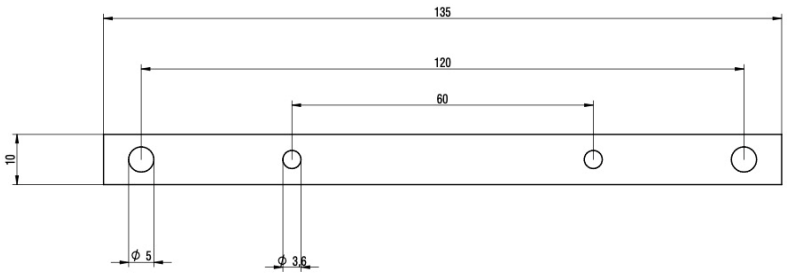
void setForward() {
  digitalWrite(in1, HIGH);
  digitalWrite(in2, LOW);
  digitalWrite(in3, HIGH);
  digitalWrite(in4, LOW);
}

void setBackward() {
  digitalWrite(in1, LOW);
  digitalWrite(in2, HIGH);
  digitalWrite(in3, LOW);
  digitalWrite(in4, HIGH);
}
```

# Appendix C - Draft, Platform



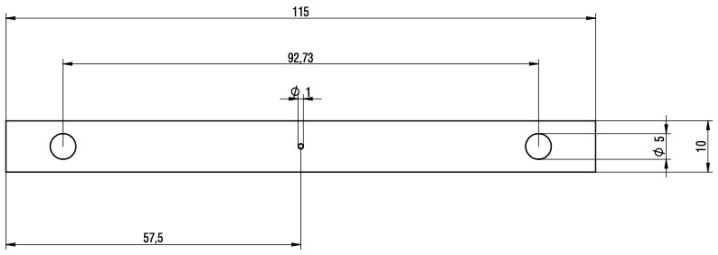
# Appendix D – Draft, Fixed frame




SOLID EDGE ACADEMIC COPY

| Detaljnr | Antal | Benämning | Material | Anmärkning |
|---|---|---|---|---|
| Skapad av: sfridell, aekholm | | Godkänd av | Material | Ersätter / Ersatt av |
| MF133X | | Fixed frame | | Datum: 2024-05-08 |
| | | | | Ritningsnr |

# Appendix E – Draft, Steering arm



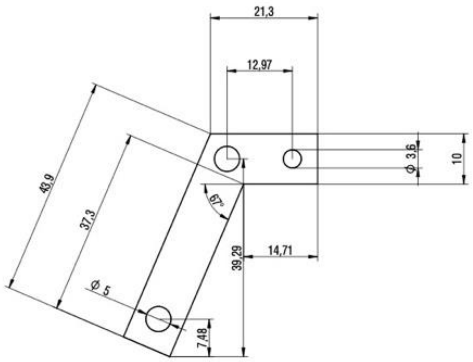

| Detaljnr | Antal | Benämning | Material | Anmärkning |
|---|---|---|---|---|
| | | | | |

| Skapad av | Godkänd av | Material | Ersätter | Ersatt av |
|---|---|---|---|---|
| sfridell, aekholm | | | | |

| | | | |
|---|---|---|---|
| MF133X | Steering arm | Datum 2024-05-08 | Ritningsnr |

SOLID EDGE ACADEMIC COPY

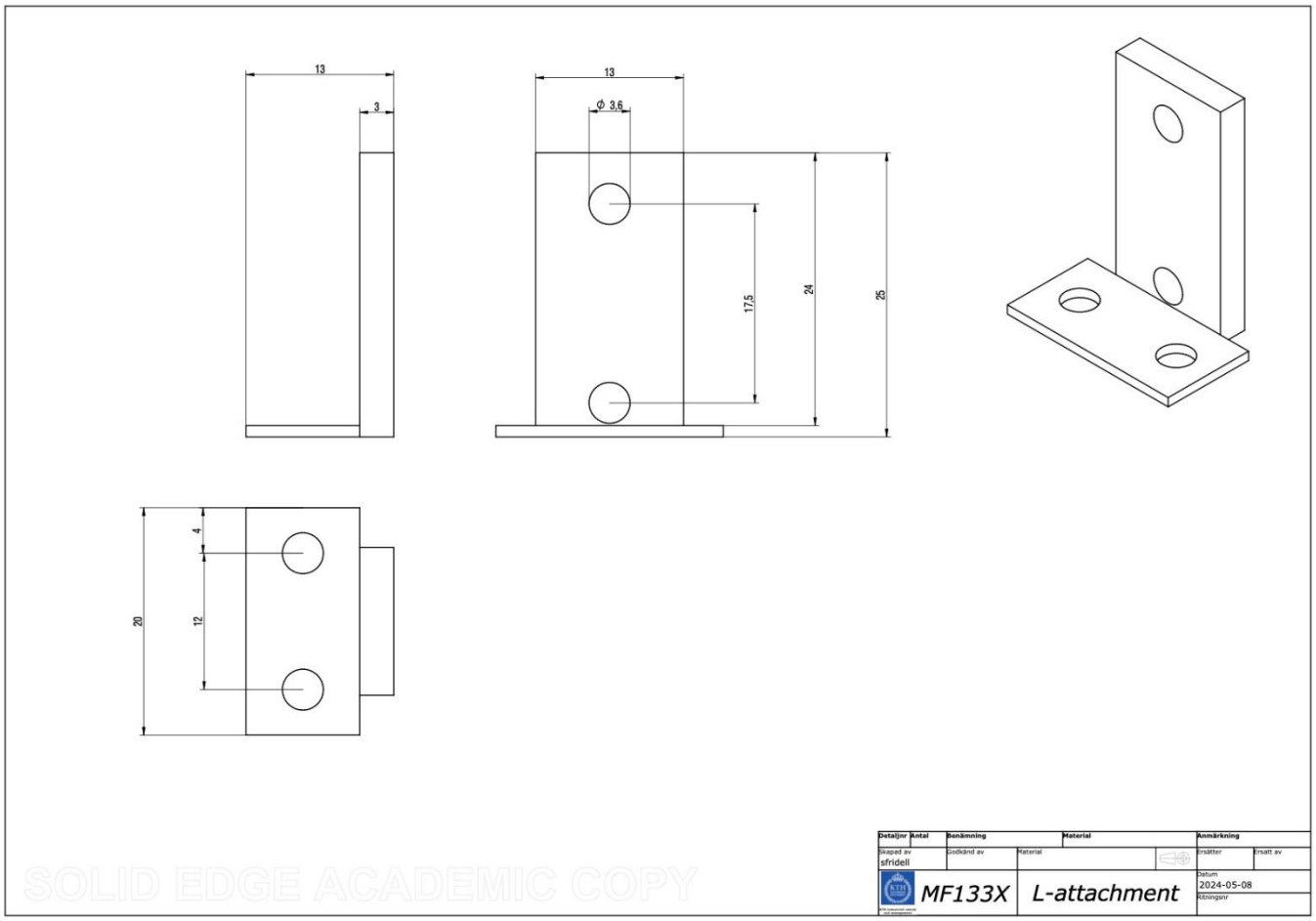
## Appendix F – Draft, Drop arm



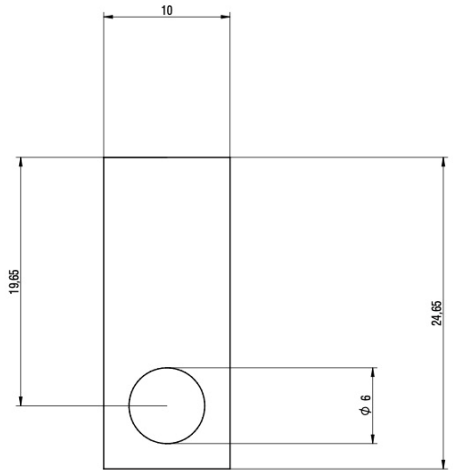

| Detaljnr | Antal | Benämning | Material | Anmärkning |
|---|---|---|---|---|
| Skapad av: sfridell, aekholm | | Godkänd av: | Material: | Ersätter / Ersatt av: |
| KTH | MF133X | Drop arm | | Datum: 2024-05-02 |
| | | | | Ritningsnr: |

SOLID EDGE ACADEMIC COPY

# Appendix G – Draft, L-attachment



# Appendix H – Draft, Wheel block




| Detaljnr | Antal | Benämning | Material | Anmärkning |
|---|---|---|---|---|
| Skapad av: sfridell, aekholm | | Godkänd av | Material | Ersätter / Ersatt av |
| MF133X | | Wheel block | | Datum: 2024-05-08 |
| | | | | Ritningsnr |

SOLID EDGE ACADEMIC COPY

## **Appendix** I **–** Angle measurements of Steering mechanism

| **Percentage of maximal hand angle [%]** | **Servo angle in code [°]** | **Measured servo angle** | **Angle left front wheel [°]** | **Angle right front wheel [°]** |
|---|---|---|---|---|
| 25 | 12.5 | 14.85 | 2.5 | 4 |
| | | 13.00 | 2 | 3 |
| | | 11.54 | 1 | 2 |
| 50 | 25 | 24.36 | 8 | 8 |
| | | 25.94 | 8 | 8 |
| | | 26.74 | 9 | 8.5 |
| 75 | 37.5 | 39.61 | 16 | 12 |
| | | 36.65 | 16.5 | 12.5 |
| | | 39.61 | 15.5 | 13 |
| 100 | 50 | 48.59 | 22.5 | 15 |
| | | 51.38 | 22 | 15.5 |
| | | 47.25 | 21.5 | 15.5 |

TRITA – ITM-EX-2024:51

www.kth.se